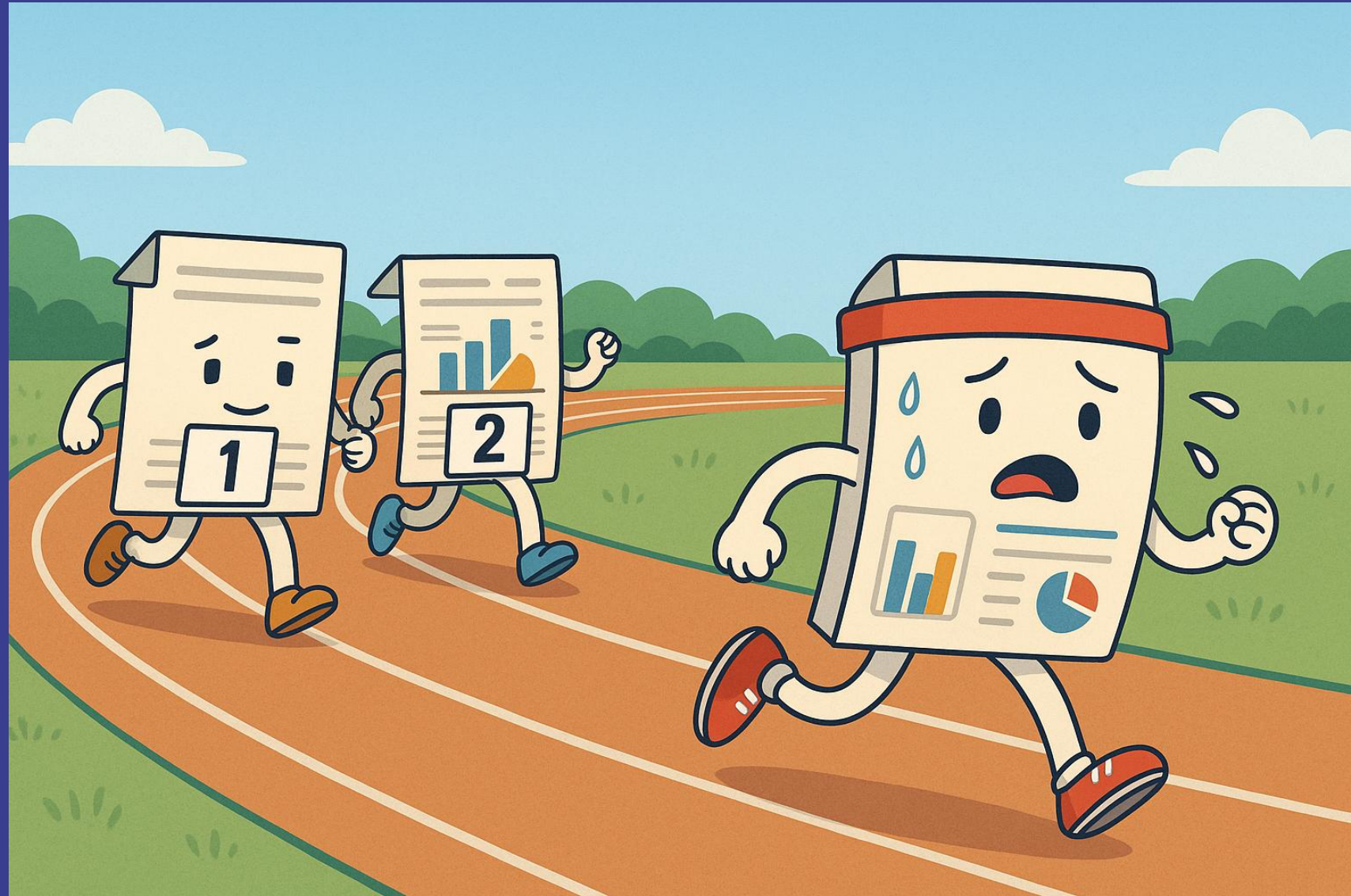


How to Diagnose a Slow Power BI Report



Jason Romans

Analytics Engineer

The DAX Shepherd



 Nashville, TN, USA

 Began Career as a SQL Server DBA

 Transitioned to Microsoft BI Stack

 Data Engineering to Data Modeling

 Infrequent Blogger

 Fan of Dimensional Models & Doctor Who

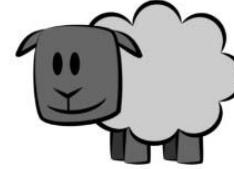


 @sql_jar

 jason-r-sql-jar

 <https://thedaxshepherd.com/>

www.thedaxshepherd.com



The DAX Shepherd

Musings on the Microsoft BI Stack



[Home](#) [About Me](#) [Simple Talk](#) [Presentations](#) [A Speaker's Journey](#)

Presentations

Sessionize

This is my [Sessionize Profile](#) that has the conferences I have spoken at along with future events. It has a couple of my most popular sessions.

Presentation Slides

This is my [GitHub Repository](#) with the presentation slides for each event.

Recorded Sessions

Simple Talks Podcast | Episode 4 – Coffee chat with Jason Romans

About Jason Romans



I love working with the Microsoft BI Stack. I am passionate about learning.

[A Speaker's Journey](#)

A large, dark grey circle with a thin white border. Inside the circle, the word "Slides" is written in a bold, white, sans-serif font.

Slides

Thank You to Our Event Sponsors

We gratefully acknowledge the generous support of our sponsors.
Their partnership makes this event possible.

Global Sponsors



Microsoft



Silver Sponsor



Bronze Sponsors



Other Sponsor

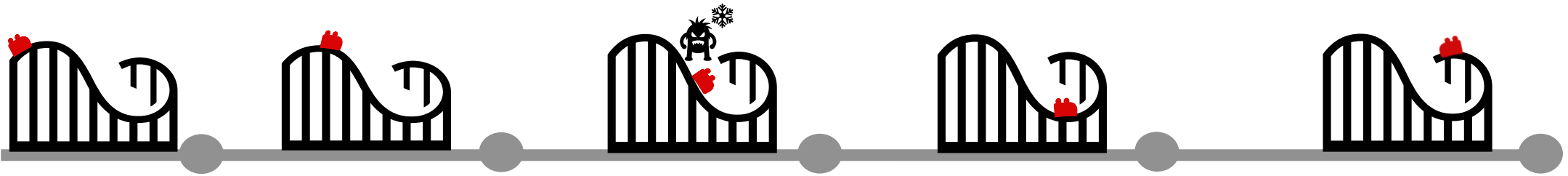


We appreciate every sponsor for helping us create a memorable event!

Shoulders of Giants

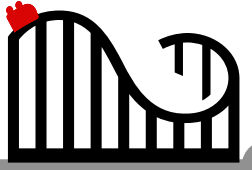


Our Journey



1. Intro
2. Performance Analyzer
3. DAX Studio
4. Tabular Editor
5. Conclusion

Our Journey



1. Intro

2. Performance Analyzer

3. DAX Studio

4. Tabular Editor

5. Conclusion

Power BI is Part of Fabric

- **If you have a Fabric capacity**
 - **Enhances what you can do with Power BI**
 - **OneLake**
 - **Warehouse & Lakehouse**
 - **Semantic Link Labs – Best Practice Analyzer**
- **What we cover applies to Power BI Pro, Power BI Premium, and Fabric**

Exam DP-600: Implementing Analytics Solutions Using Microsoft Fabric

Exam DP-600: Implementing Analytics Solutions Using Microsoft Fabric

Design and build semantic models

- Identify use cases for DAX Studio and Tabular Editor 2

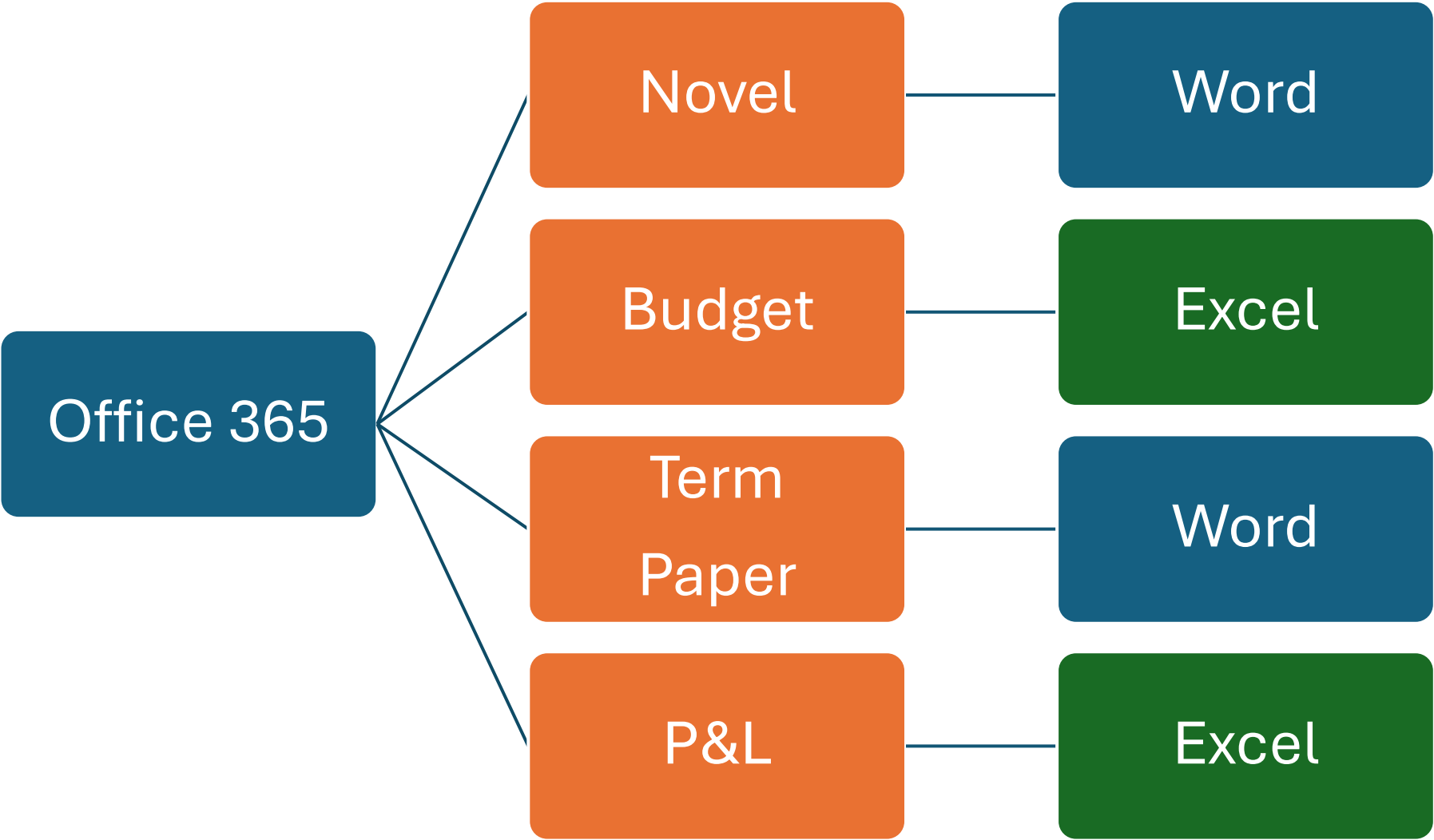
Optimize enterprise-scale semantic models

- Improve DAX performance by using DAX Studio
- Optimize a semantic model by using Tabular Editor 2

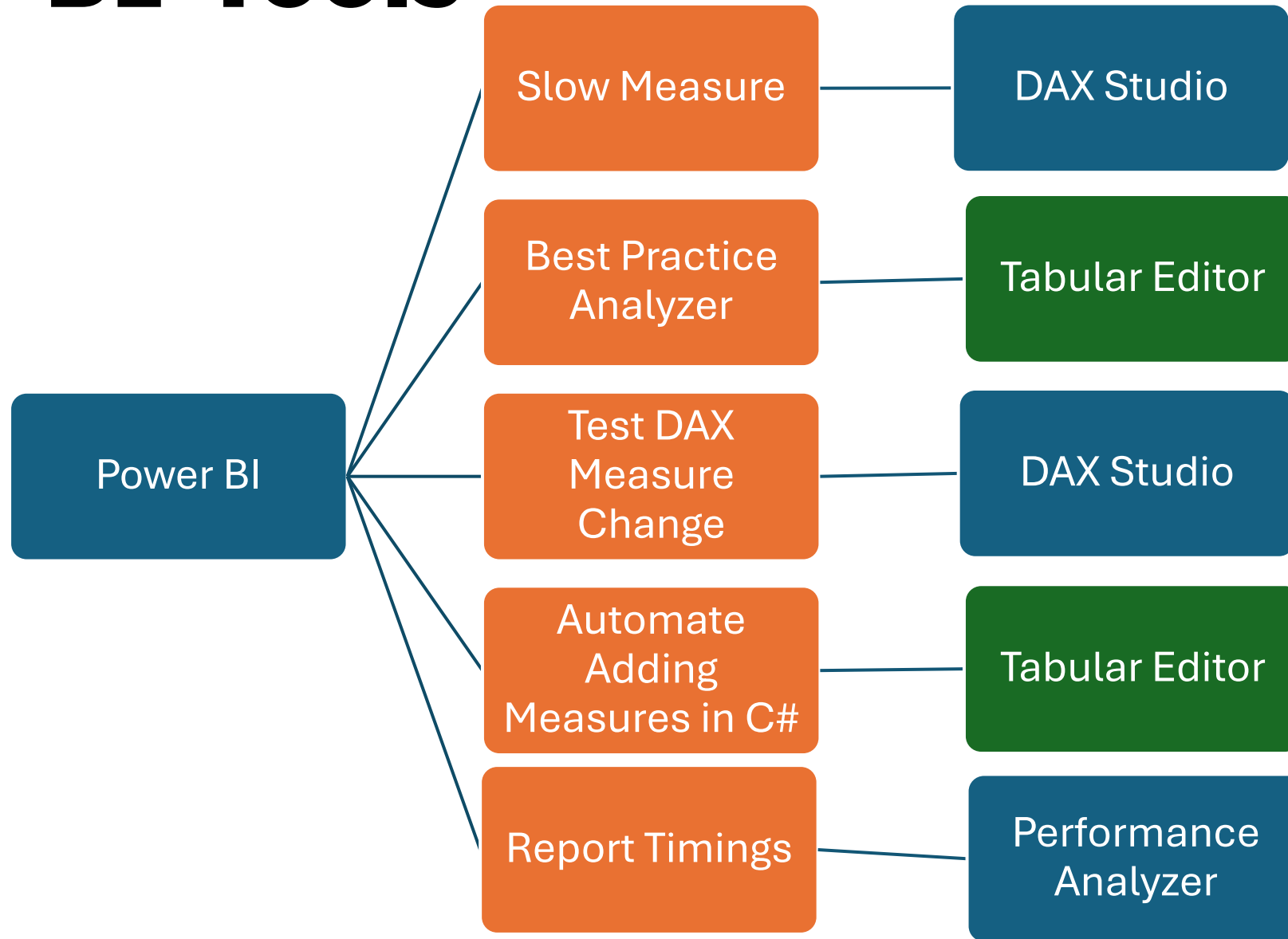
Why is my report slow?

- Motivating Factor
- This can have a lasting impact
 - Power BI Developer Skills
 - Career Development

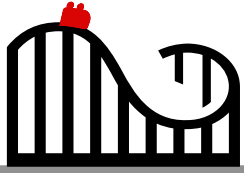
Office 365



Power BI Tools



Our Journey



1. Intro

2. Performance Analyzer

3. DAX Studio

4. Tabular Editor

5. Conclusion

Performance Analyzer

- Not an External Tool
- Built into Power BI Desktop

Performance Analyzer on the web

- October Power BI Release Notes
 - This functionality will be available to your tenant in the next few weeks.

Performance Analyzer

File



Pause visuals

Refresh visuals

Queries

Home



Optimization presets ▾

Report

Insert



Performance analyzer

Review

Modeling



Apply all slicers button

Apply

View



Performance analyzer

Review

Optimize

Help

External tools

Performance Analyzer

Start Recording

[illegible]

Performance Analyzer Recording

Performance analyzer

▶ Start recording

↺ Refresh visuals

⏹ Stop

◇ Clear

📄 Export

Name	Duration (ms) ↓
🕒 Recording started (5/1/2...	-
Interact with your report t...	-

Performance Analyzer

Refresh Visuals

Performance analyzer >>

▶ Start recording ↻ Refresh visuals ⏹ Stop

🧼 Clear 📄 Export

Name	Duration (ms) ↓
🕒 Recording started (5/1/2...	-
Interact with your report t...	-

Individual Refresh Button

Note: Values for Other will be different

Date	Total Quantity				
Wednesday, January 01, 2020	7,759				
Thursday, January 02, 2020	8,256				
Friday, January 03, 2020	5,482				
Saturday, January 04, 2020	8,608				
Sunday, January 05, 2020	1,144				
Monday, January 06, 2020	3,823				
Tuesday, January 07, 2020	4,414				

List of Visuals

Performance analyzer >>

▶ Start recording ↺ Refresh visuals ⏻ Stop

🧼 Clear 📄 Export

Name	Duration (ms) ↓
↺ Refreshed visual	-
⊕ Card	65
⊕ Card	66
⊕ Table	102

Expanded View

Performance analyzer >>

▶ Start recording ↻ Refresh visuals ⏹ Stop

⧫ Clear 📄 Export

Name	Duration (ms) ↓
↻ Refreshed visual	-
☐ Card	65
DAX query	3
Visual display	3
Other	59
📄 Copy query	
📄 Run in DAX Query View	
☐ Card	66
DAX query	3
Visual display	4
Other	59
📄 Copy query	
📄 Run in DAX Query View	
☐ Table	102
DAX query	29
Visual display	31
Other	42
📄 Copy query	
📄 Run in DAX Query View	

Detail on Card

 Refreshed visual	-
 Card	65
DAX query	3
Visual display	3
Other	59
 Copy query	
 Run in DAX Query View	

This just in...

- Field Parameters added to Performance Analyzer

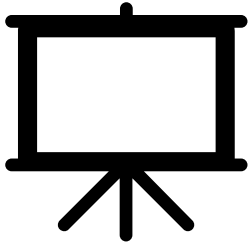
Field Parameters

Year	Margin	Sales Amount	MYFieldParameters
 2010	33,823,854.13	60,104,007.31	 Margin
 2011	59,624,049.16	106,369,558.11	 Sales Amount
 2012	70,574,206.47	126,526,827.08	
 2013	115,530,028.15	207,035,054.38	 Total Quantity
 2014	165,136,883.36	296,225,092.53	
 2015	133,147,101.44	238,734,404.74	
 2016	102,106,742.83	182,718,147.59	
 2017	172,816,713.92	309,617,243.26	
 2018	222,188,888.51	397,466,246.66	
 2019	157,047,924.81	281,077,320.03	
 2020	35,053,415.89	62,598,972.44	
Jan	16,480,966.75	29,393,144.47	
Feb	17,876,583.26	31,961,657.30	
Mar	695,865.88	1,244,170.67	
Total	1,267,049,808.67	2,268,472,874.13	

Perf Analyzer – Field Parameters

 Matrix	51
DAX query	4
Evaluated parameters	13
Visual display	22
Other	12
 Copy query	
 Run in DAX query view	

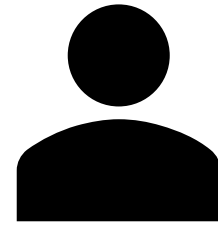
3 Numbers



Visual Display



Other



DAX Query

Visual Display

🔄 Refreshed visual	-
📄 Card	65
DAX query	3
Visual display	3
Other	59
📄 Copy query	
📄 Run in DAX Query View	

- Visual Display
 - Time spent on producing the visual
- Restaurant
 - Plating

Slow Visual



Performance analyzer

Start recording Refresh visuals Stop

Clear Export

Name	Duration (ms) ↓
Refreshed visual	-
Margin and Margin % by Date	725
Total Cost by Date	724
Sales Amount by Date	152
Margin %, Margin, Sales Amount, Total Co...	723
DAX query	10
Visual display	526
Other	187
Copy query	
Run in DAX Query View	
Sales Amount by Date, Brand and Age	927
Total Cost and Total Quantity by Date	722

Visual Display - Solutions

- Reduce the complexity of visual
 - Granularity of visual
- Use a Background Image
- More in Resources

Other

🔄 Refreshed visual	-
📄 Card	65
DAX query	3
Visual display	3
Other	59
📄 Copy query	
📄 Run in DAX Query View	

- Other
 - Time waiting until DAX Query can be executed
- Restaurant
 - Waiting to order

Other - Solutions

- Reduce Visualizations on the Page
 - Scrollbar?
- Evaluate where the bottleneck is
 - May be a combination of other factors

DAX Query

🔄 Refreshed visual	-
📄 Card	65
DAX query	3
Visual display	3
Other	59
📄 Copy query	
📄 Run in DAX Query View	

- DAX Query
 - Time it takes to execute the DAX query
- Restaurant
 - Time to make the food

Slow DAX Query

Performance analyzer >>

Start recording Refresh visuals Stop

Clear Export

Name	Duration (ms) ↓
Refreshed visual	-
+ Card	52
+ Card	52
- Table	28059
DAX query	27992
Visual display	30
Other	37
Copy query	
Run in DAX Query View	

DAX Query - Solutions



Refreshed visual	-
Card	65
DAX query	3
Visual display	3
Other	59
Copy query	
Run in DAX Query View	

- Investigate query
 - Where is most of the query time spent
- Can the DAX code be optimized?
 - Rewrite the DAX code

Run in DAX Query View

	Table	102
	DAX query	29
	Visual display	31
	Other	42
	Copy query	
	Run in DAX Query View	

Query View

- Queries are saved with the model
- Share or preserve slow query
- First exposure to DAX Queries
- Has many uses – i.e. validations
- Not good for diagnosing slow queries

DAX Query View for Web

- <https://powerbi.microsoft.com/en-us/blog/deep-dive-into-dax-query-view-for-web/>
- Use against models in Workspace

Power BI Demo Performance Analyzer

Performance Analyzer

- Informed Decisions
- Don't underestimate this tool
- You can isolate where the issue is
- Why spend all day optimizing DAX if it isn't the issue

What about that slow DAX Query?

Performance analyzer >>

Start recording Refresh visuals Stop

Clear Export

Name	Duration (ms) ↓
Refreshed visual	-
Table	61668
DAX query	61617
Visual display	23
Other	28
Copy query	
Run in DAX Query View	

External Tools



Installation

- Full Install
- Power BI Desktop
- External Tools Tab

External Tools Tab

File

Home

Insert

Modeling

View

Optimize

Help

External tools

Format

Data / Drill



ALM Toolkit



Bravo



Model Documenter



DAX Studio

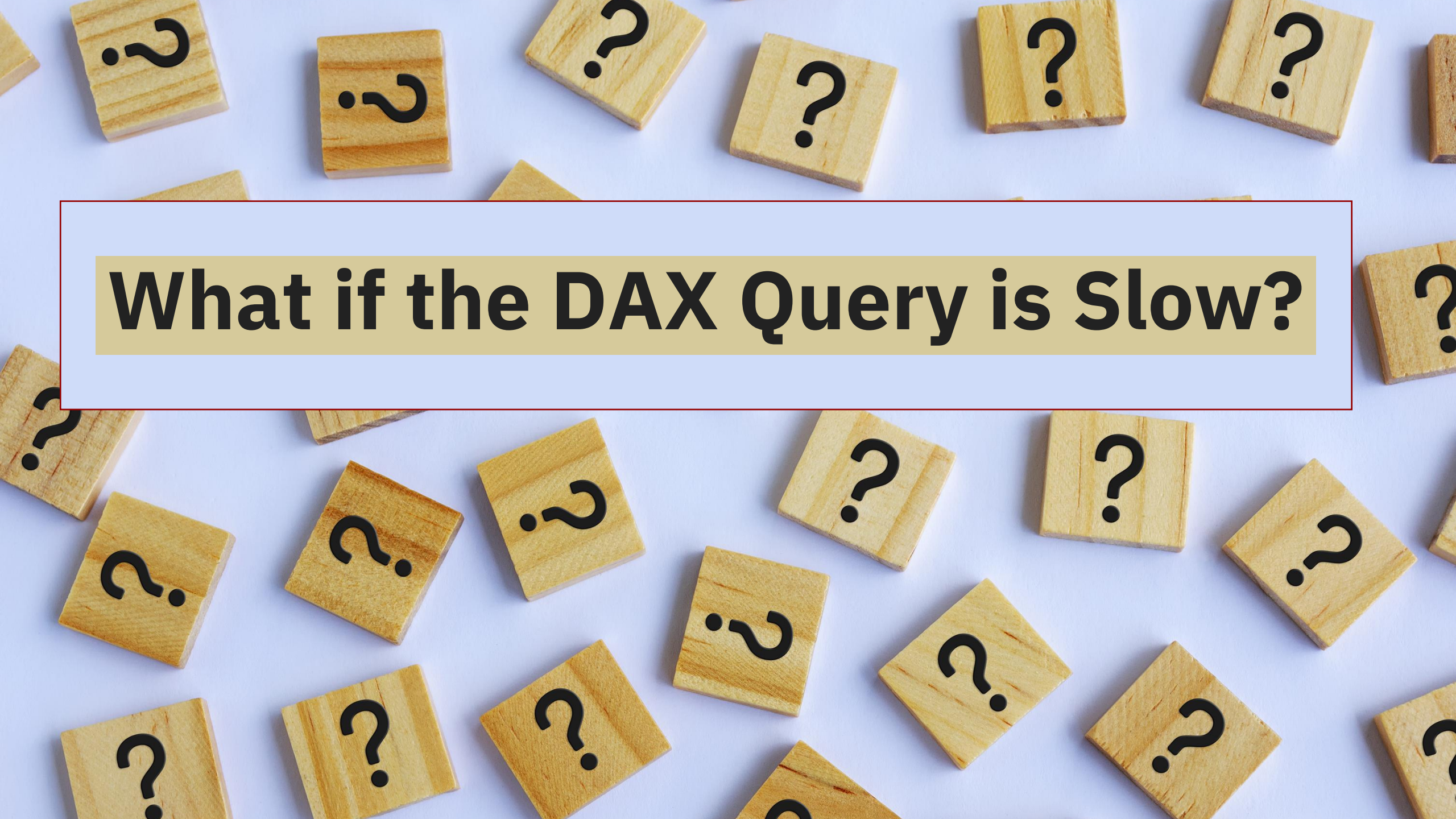


Tabular Editor



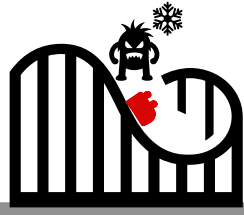
Tabular Editor 3

External tools

The background of the image is a light blue surface covered with numerous small, square wooden blocks. Each block has a black question mark printed on its top face. The blocks are scattered across the entire frame, some overlapping and others separate, creating a pattern of question marks. In the center of the image, there is a white rectangular box with a thin red border. Inside this box, the text "What if the DAX Query is Slow?" is written in a bold, black, sans-serif font. The text is centered both horizontally and vertically within the box.

What if the DAX Query is Slow?

Our Journey



1. Intro
2. Performance Analyzer
- 3. DAX Studio**
4. Tabular Editor
5. Conclusion



DAX

DAX Studio

- Download
 - <https://daxstudio.org/>
- Author Measures and Queries
- Connect to Semantic Models
- Performance Tuning
 - Details on how the query is processed

Slow DAX Query

Performance analyzer >>

▶ Start recording ↺ Refresh visuals ⏹ Stop

⧫ Clear 📄 Export

Name	Duration (ms) ↓
↺ Refreshed visual	-
📄 Table	61668
DAX query	61617
Visual display	23
Other	28
📄 Copy query	
📄 DAX Run in DAX Query View	

Copy query

Performance analyzer		>>
▶ Start recording		↺ Refresh visuals ⏹ Stop
		⧻ Clear 📄 Export
Name	Duration (ms) ↓	
↺ Refreshed visual	-	
📄 Table	61668	
DAX query	61617	
Visual display	23	
Other	28	
📄 Copy query		
📄 Run in DAX Query View		

Copied

Performance analyzer

▶ Start recording

↺ Refresh visuals

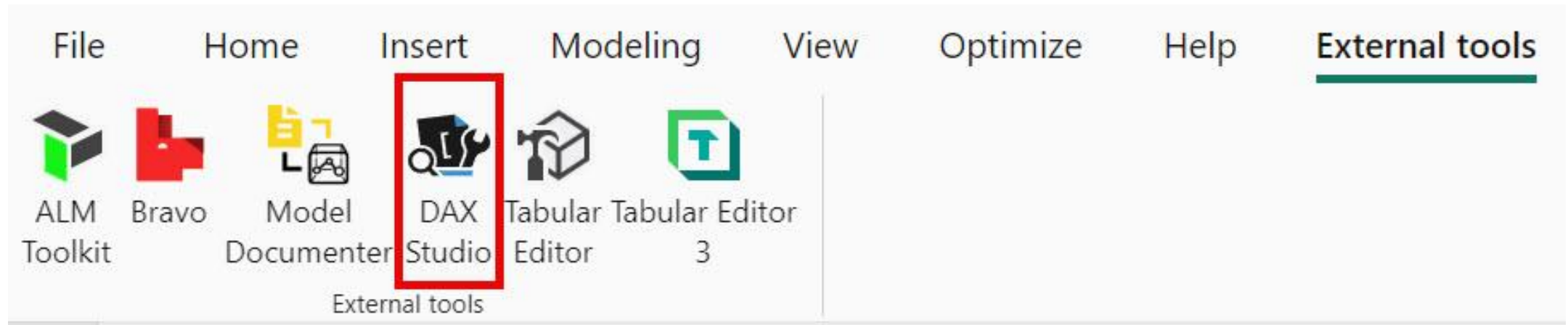
⊞ Stop

◇ Clear

📄 Export

Name	Duration (ms) ↓
↺ Refreshed visual	-
☐ Table	61668
DAX query	61617
Visual display	23
Other	28
✓ Copied	
📄 Run in DAX Query View	

Power BI - Open DAX Studio



Paste in DAX Studio

DAX Studio - 3.0.11

File Home Advanced Help

Run Cancel Query Builder Clear Cache Clear on Run Results Cut Copy Paste Undo Redo Format Query To Upper To Lower Debug Commas Comment Uncomment Merge XML Find Replace Load Perf Data All Queries Query Plan Server Timings Connect Refresh Metadata

Metadata Functions DMV

Contoso1M_Docker_BPATest Model

Search

- Customer
- Date
- Product
- Sales
- Store

```
1 DEFINE
2   VAR __DS0Core =
3     SUMMARIZECOLUMNS(
4       ROLLUPADDSUBTOTAL(ROLLUPGROUP('Product'[Category], 'Product'[Brand]), "IsGrandTotalRowTotal"),
5       "Margin", 'Sales'[Margin]
6     )
7
8   VAR __DS0Primarywindowed =
9     TOPN(502, __DS0Core, [IsGrandTotalRowTotal], 0, 'Product'[Category], 1, 'Product'[Brand], 1)
10
11 EVALUATE
12   __DS0Primarywindowed
13
14 ORDER BY
15   [IsGrandTotalRowTotal] DESC, 'Product'[Category], 'Product'[Brand]
```

Log Results History

	Start	Duration ⓘ	Message
ⓘ	16:34:01		Establishing Connection
ⓘ	16:34:01		Connected

Let's Run It

DAX Studio - 3.0.11

File **Home** Advanced Help

Run Cancel Query Builder Clear Cache Clear on Run Results Output Edit Format Query To Upper To Lower Debug Commas Comment Uncomment Merge XML Find Replace Load Perf Data All Queries Query Plan Server Timings Connect Refresh Metadata

Metadata Functions DMV

Contoso1M_Docker_BPATest Model

Search

- Customer
- Date
- Product
- Sales
- Store

```
1 DEFINE
2   VAR __DS0Core =
3     SUMMARIZECOLUMNS(
4       ROLLUPADDSUBTOTAL(ROLLUPGROUP('Product'[Category], 'Product'[Brand]), "IsGrandTotalRowTotal"),
5       "Margin", 'Sales'[Margin]
6     )
7
8   VAR __DS0Primarywindowed =
9     TOPN(502, __DS0Core, [IsGrandTotalRowTotal], 0, 'Product'[Category], 1, 'Product'[Brand], 1)
10
11 EVALUATE
12   __DS0Primarywindowed
13
14 ORDER BY
15   [IsGrandTotalRowTotal] DESC, 'Product'[Category], 'Product'[Brand]
```

Log Results History

	Start	Duration ⓘ	Message
ⓘ	16:34:01		Establishing Connection
ⓘ	16:34:01		Connected

Results – Like Query View

FileHomeAdvancedHelp

Run

Cancel

Query Builder

Clear Cache

Clear on Run

Results

Cut

Copy

Paste

Format Query

A To Upper

a To Lower

Debug Commas

Comment

Uncomment

Merge XML

Find

Replace

Load Perf Data

All Queries

Query Plan

Server Timings

Connect

Refresh Metadata

QueryCacheOutputEditFormatFindPower BITracesConnection

MetadataFunctionsDMV

Contoso1M_Docker_BPATest

Model

Search

Customer

Date

Product

Sales

Store

```
1 DEFINE
2   VAR __DS0Core =
3     SUMMARIZECOLUMNS(
4       ROLLUPADISSUBTOTAL(ROLLUPGROUP('Product'[Category], 'Product'[Brand]), "IsGrandTotalRowTotal"),
5       "Margin", 'Sales'[Margin]
6     )
7
8   VAR __DS0Primarywindowed =
9     TOPN(502, __DS0Core, [IsGrandTotalRowTotal], 0, 'Product'[Category], 1, 'Product'[Brand], 1)
10
11 EVALUATE
12   __DS0Primarywindowed
13
14 ORDER BY
15   [IsGrandTotalRowTotal] DESC, 'Product'[Category], 'Product'[Brand]
```

LogResultsHistoryServer Timings

Category	Brand	IsGrandTotalRowTotal	Margin
		True	1267049808.6663
Audio	Contoso	False	5547636.5545
Audio	Northwind Traders	False	5276681.882
Audio	Wide World Importers	False	21868862.1847
Cameras and camcorders	A. Datum	False	28002980.6738
Cameras and camcorders	Contoso	False	19656707.4536
Cameras and camcorders	Fabrikam	False	71111794.4404
Cell phones	Contoso	False	22490172.075
Cell phones	The Phone Company	False	174204193.028
Computers	Adventure Works	False	189547913.3088

Query1.dax* X

Ready

I Ln 15, Col 50localhost:5319316.0.126.2710230 rows00:00.0

Traces

File

Run

▼

Cancel

Query

Query Builder

Clear Cache

Cache

Clear on Run

Cache

Results

▼

Output

Cut

Copy

Paste

Edit

Undo

Redo

Format Query

▼

Format

To Upper

To Lower

Debug Commas

Format

Comment

Uncomment

Merge XML

Find

Replace

Find

Load Perf Data

Power BI

All Queries

Traces

Query Plan

Traces

Server Timings

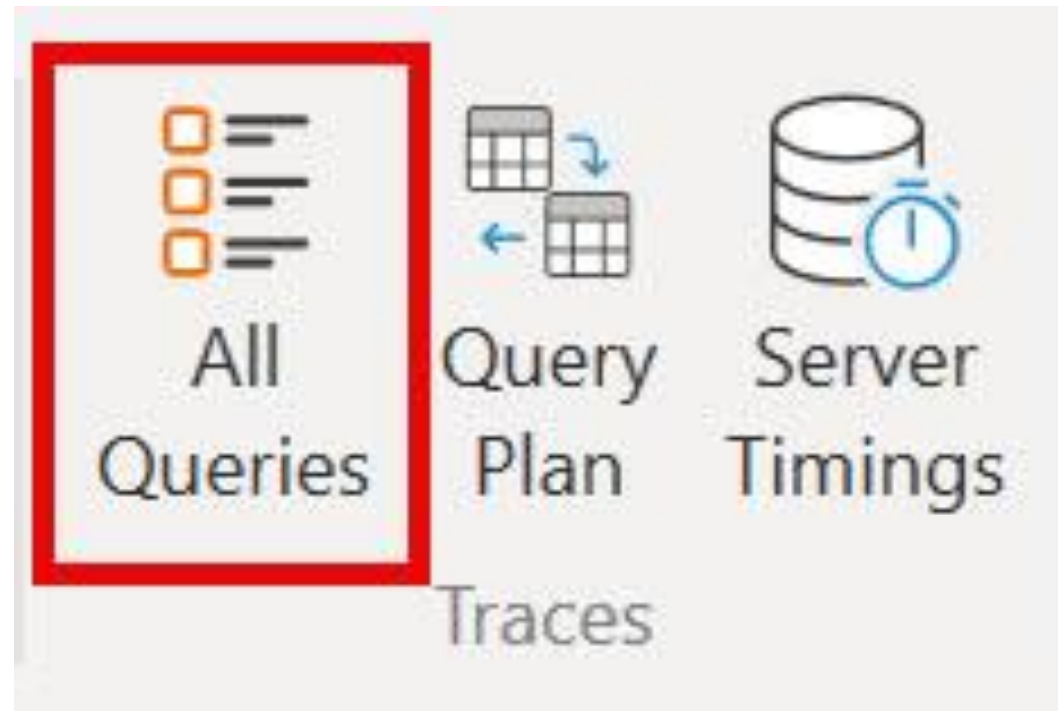
Traces

Connect

Refresh Metadata

Connection

DAX Studio – All Queries



Like SQL Server Profiler

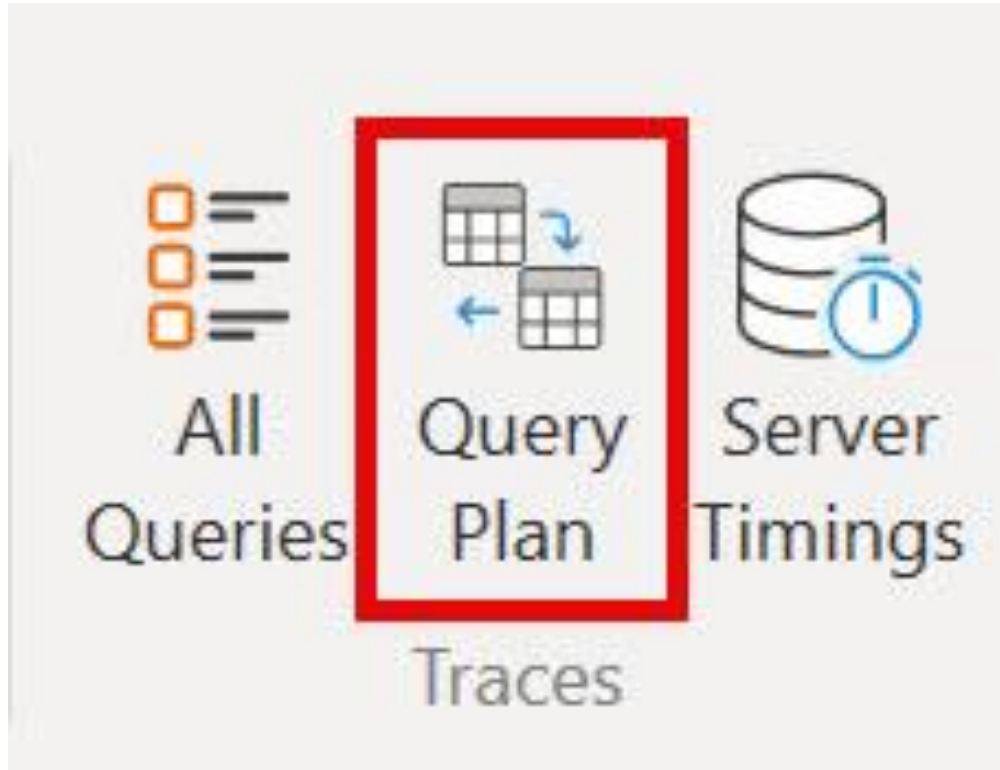
DAX Studio – All Queries

LogResultsHistoryAll Queries

RecordPauseStopClearCopyExportInfoFilters

StartTime	Type	Duration	User	Database	Query
10:12:54	DAX	10ms	MSI\roman	Contoso1M_Docker_BPATest	DEFINE VAR __DS0Core = SUMMARIZECOLUMNS(ROLLUPADDISSUBTOTAL(ROLLUPGROUP('Product'[Category], 'Product'[Brand]), "IsGrandTotalRowTotal"), "Margin", 'Sa...
10:12:51	DAX	0ms	MSI\roman	Contoso1M_Docker_BPATest	DEFINE VAR __DS0Core = SUMMARIZE('Product', 'Product'[Category], 'Product'[Brand]) VAR __DS0PrimaryWindowed = TOPN(501, __DS0Core, 'Product'[Category], 1, 'Pro...
10:12:50	DAX	2ms	MSI\roman	Contoso1M_Docker_BPATest	DEFINE VAR __DS0Core = DISTINCT('Product'[Category]) VAR __DS0PrimaryWindowed = TOPN(501, __DS0Core, 'Product'[Category], 1) EVALUATE __DS0PrimaryWindowe...
10:11:45	DAX	187ms	MSI\roman	Contoso1M_Docker_BPATest	DEFINE VAR __DS0Core = SUMMARIZECOLUMNS(ROLLUPADDISSUBTOTAL(ROLLUPGROUP('Date'[Year Month], 'Product'[Brand]), "IsGrandTotalRowTotal"), "Total_BIG_Or...
10:11:39	DAX	9ms	MSI\roman	Contoso1M_Docker_BPATest	DEFINE VAR __DS0Core = SELECTCOLUMNS(KEEPFILTERS(FILTER(KEEPFILTERS(SUMMARIZECOLUMNS('Date'[Year Month], 'Product'[Brand], "CountRowsSales", COUNT...
10:11:24	DAX	6ms	MSI\roman	Contoso1M_Docker_BPATest	EVALUATE ROW("MinBrand", CALCULATE(MIN('Product'[Brand])))
10:10:33	DAX	31,984ms	MSI\roman	Contoso1M_Docker_BPATest	DEFINE VAR __DS0Core = SUMMARIZECOLUMNS(ROLLUPADDISSUBTOTAL(ROLLUPGROUP('Date'[Year Month], 'Product'[Brand]), "IsGrandTotalRowTotal"), "Total_BIG_Or...

Query Plan



Query Plans

Two Types

- Logical
- Physical

DAX Studio – Query Plan

Log	Results	History	Server Timings	Query Plan
Record Pause Stop Clear Export Info				
Line	Records	Physical Query Plan		
1		PartitionIntoGroups: IterPhyOp LogOp=Order IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]") #Groups=1 #Rows=120		
2	1	AggregationSpool<Order>: SpoolPhyOp #Records=1		
3		Proxy: IterPhyOp LogOp=TableVarProxy IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]")		
4		Proxy: IterPhyOp LogOp=TableVarProxy IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]")		
5		Union: IterPhyOp LogOp=Union IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]")		
6		GroupSemijoin: IterPhyOp LogOp=GroupSemijoin IterCols(0, 1, 2)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good])		
7	119	Spool_Iterator<SpoolIterator>: IterPhyOp LogOp=DistinctCount_Vertipaq IterCols(0)('Date'[Year Month]) #Records=119 #KeyCols=70 #ValueCols=1		
8	119	ProjectionSpool<ProjectFusion<Copy>>: SpoolPhyOp #Records=119		
9		Cache: IterPhyOp #FieldCols=1 #ValueCols=1		
10		GroupSemijoin: IterPhyOp LogOp=GroupSemijoin IterCols(0, 1, 2)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good])		
11	1	Spool_Iterator<SpoolIterator>: IterPhyOp LogOp=DistinctCount_Vertipaq IterCols(0)('Date'[Year Month]) #Records=1 #KeyCols=70 #ValueCols=1		
Line		Logical Query Plan		
1		_DS0Core: Union: RelLogOp VarName=__DS0Core DependOnCols() 0-3 RequiredCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]")		
2		GroupSemijoin: RelLogOp DependOnCols() 0-2 RequiredCols(0, 1, 2)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good])		
3		Scan_Vertipaq: RelLogOp DependOnCols() 0-0 RequiredCols(0)('Date'[Year Month])		
4		Constant: ScaLogOp DependOnCols() Boolean DominantValue=false		
5		Calculate: ScaLogOp MeasureRef=[Total BIG Orders Good] DependOnCols(0)('Date'[Year Month]) Integer DominantValue=BLANK		
6		DistinctCount_Vertipaq: ScaLogOp DependOnCols(0)('Date'[Year Month]) Integer DominantValue=BLANK		
7		Scan_Vertipaq: RelLogOp DependOnCols(0)('Date'[Year Month]) 2-2 RequiredCols(0)('Date'[Year Month])		
8		Filter_Vertipaq: RelLogOp DependOnCols(0) 1-1 RequiredCols(1)('Sales'[Net Price])		
9		Scan_Vertipaq: RelLogOp DependOnCols(0) 1-1 RequiredCols(1)('Sales'[Net Price])		
10		GreaterThan: ScaLogOp DependOnCols(1)('Sales'[Net Price]) Boolean DominantValue=NONE		
11		'Sales'[Net Price]: ScaLogOp DependOnCols(1)('Sales'[Net Price]) Currency DominantValue=NONE		

Query Plan – One Useful Thing

LogResultsHistoryServer TimingsQuery Plan

RecordPauseStopClearExportInfo

Line	Records	Physical Query Plan
1		PartitionIntoGroups: IterPhyOp LogOp=Order IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal], "[Total_BIG_Orders_Good], "[[]] #Groups=1 #Rows=120
2	1	AggregationSpool<Order>: SpoolPhyOp #Records=1
3		Proxy: IterPhyOp LogOp=TableVarProxy IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal], "[Total_BIG_Orders_Good], "[[]]
4		Proxy: IterPhyOp LogOp=TableVarProxy IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal], "[Total_BIG_Orders_Good], "[[]]
5		Union: IterPhyOp LogOp=Union IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal], "[Total_BIG_Orders_Good], "[[]]
6		GroupSemiJoin: IterPhyOp LogOp=GroupSemiJoin IterCols(0, 1, 2)('Date'[Year Month], "[IsGrandTotalRowTotal], "[Total_BIG_Orders_Good])
7	119	Spool_Iterator<SpoolIterator>: IterPhyOp LogOp=DistinctCount_Vertipaq IterCols(0)('Date'[Year Month]) #Records=119 #KeyCols=70 #ValueCols=1
8	119	ProjectionSpool<ProjectFusion<Copy>>: SpoolPhyOp #Records=119
9		Cache: IterPhyOp #FieldCols=1 #ValueCols=1
10		GroupSemiJoin: IterPhyOp LogOp=GroupSemiJoin IterCols(0, 1, 2)('Date'[Year Month], "[IsGrandTotalRowTotal], "[Total_BIG_Orders_Good])
11	1	Spool_Iterator<SpoolIterator>: IterPhyOp LogOp=DistinctCount_Vertipaq #Records=1 #KeyCols=70 #ValueCols=1

Line	Logical Query Plan
1	_DS0Core: Union: RelLogOp VarName=__DS0Core DependOnCols() 0-3 RequiredCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal], "[Total_BIG_Orders_Good], "[[]]
2	GroupSemiJoin: RelLogOp DependOnCols() 0-2 RequiredCols(0, 1, 2)('Date'[Year Month], "[IsGrandTotalRowTotal], "[Total_BIG_Orders_Good])
3	Scan_Vertipaq: RelLogOp DependOnCols() 0-0 RequiredCols(0)('Date'[Year Month])
4	Constant: ScaLogOp DependOnCols() Boolean DominantValue=false
5	Calculate: ScaLogOp MeasureRef=[Total BIG Orders Good] DependOnCols(0)('Date'[Year Month]) Integer DominantValue=BLANK
6	DistinctCount_Vertipaq: ScaLogOp DependOnCols(0)('Date'[Year Month]) Integer DominantValue=BLANK
7	Scan_Vertipaq: RelLogOp DependOnCols(0)('Date'[Year Month]) 2-2 RequiredCols(0)('Date'[Year Month])
8	Filter_Vertipaq: RelLogOp DependOnCols() 1-1 RequiredCols(1)('Sales'[Net Price])
9	Scan_Vertipaq: RelLogOp DependOnCols() 1-1 RequiredCols(1)('Sales'[Net Price])
10	GreaterThan: ScaLogOp DependOnCols(1)('Sales'[Net Price]) Boolean DominantValue=NONE
11	'Sales'[Net Price]: ScaLogOp DependOnCols(1)('Sales'[Net Price]) Currency DominantValue=NONE

DAX Studio – Records

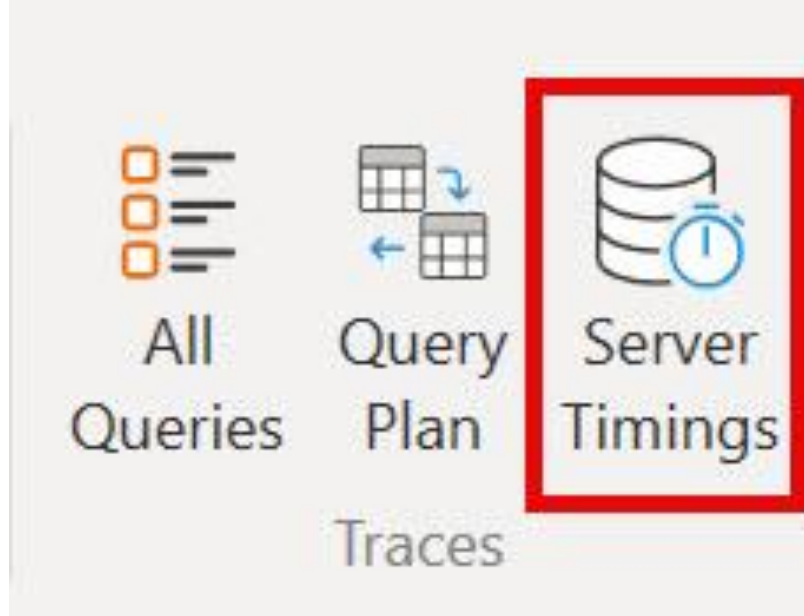
LogResultsHistoryServer TimingsQuery Plan

RecordPauseStopClearExportInfo

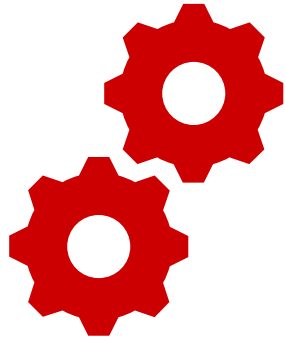
Line	Records	Physical Query Plan
1		PartitionIntoGroups: IterPhyOp LogOp=Order IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]") #Groups=1 #Rows=120
2	1	AggregationSpool<Order>: SpoolPhyOp #Records=1
3		Proxy: IterPhyOp LogOp=TableVarProxy IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]")
4		Proxy: IterPhyOp LogOp=TableVarProxy IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]")
5		Union: IterPhyOp LogOp=Union IterCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]")
6		GroupSemijoin: IterPhyOp LogOp=GroupSemijoin IterCols(0, 1, 2)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good])
7	119	Spool_Iterator<SpoolIterator>: IterPhyOp LogOp=DistinctCount_Vertipaq IterCols(0)('Date'[Year Month]) #Records=119 #KeyCols=70 #ValueCols=1
8	119	ProjectionSpool<ProjectFusion<Copy>>: SpoolPhyOp #Records=119
9		Cache: IterPhyOp #FieldCols=1 #ValueCols=1
10		GroupSemijoin: IterPhyOp LogOp=GroupSemijoin IterCols(0, 1, 2)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good])
11	1	Spool_Iterator<SpoolIterator>: IterPhyOp LogOp=DistinctCount_Vertipaq IterCols(0)('Date'[Year Month]) #Records=1 #KeyCols=70 #ValueCols=1

Line	Logical Query Plan
1	_DS0Core: Union: RelLogOp VarName=__DS0Core DependOnCols() 0-3 RequiredCols(0, 1, 2, 3)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good]", "[[]]")
2	GroupSemiJoin: RelLogOp DependOnCols() 0-2 RequiredCols(0, 1, 2)('Date'[Year Month], "[IsGrandTotalRowTotal]", "[Total_BIG_Orders_Good])
3	Scan_Vertipaq: RelLogOp DependOnCols() 0-0 RequiredCols(0)('Date'[Year Month])
4	Constant: ScaLogOp DependOnCols() Boolean DominantValue=false
5	Calculate: ScaLogOp MeasureRef=[Total BIG Orders Good] DependOnCols(0)('Date'[Year Month]) Integer DominantValue=BLANK
6	DistinctCount_Vertipaq: ScaLogOp DependOnCols(0)('Date'[Year Month]) Integer DominantValue=BLANK
7	Scan_Vertipaq: RelLogOp DependOnCols(0)('Date'[Year Month]) 2-2 RequiredCols(0)('Date'[Year Month])
8	Filter_Vertipaq: RelLogOp DependOnCols() 1-1 RequiredCols(1)('Sales'[Net Price])
9	Scan_Vertipaq: RelLogOp DependOnCols() 1-1 RequiredCols(1)('Sales'[Net Price])
10	GreaterThan: ScaLogOp DependOnCols(1)('Sales'[Net Price]) Boolean DominantValue=NONE
11	'Sales'[Net Price]: ScaLogOp DependOnCols(1)('Sales'[Net Price]) Currency DominantValue=NONE

DAX Studio – Server Timings



A Tale of Two Engines



Formula Engine



Storage Engine

Formula Engine

- Conductor
- Does not cache
- Single threaded
- Complex Operations

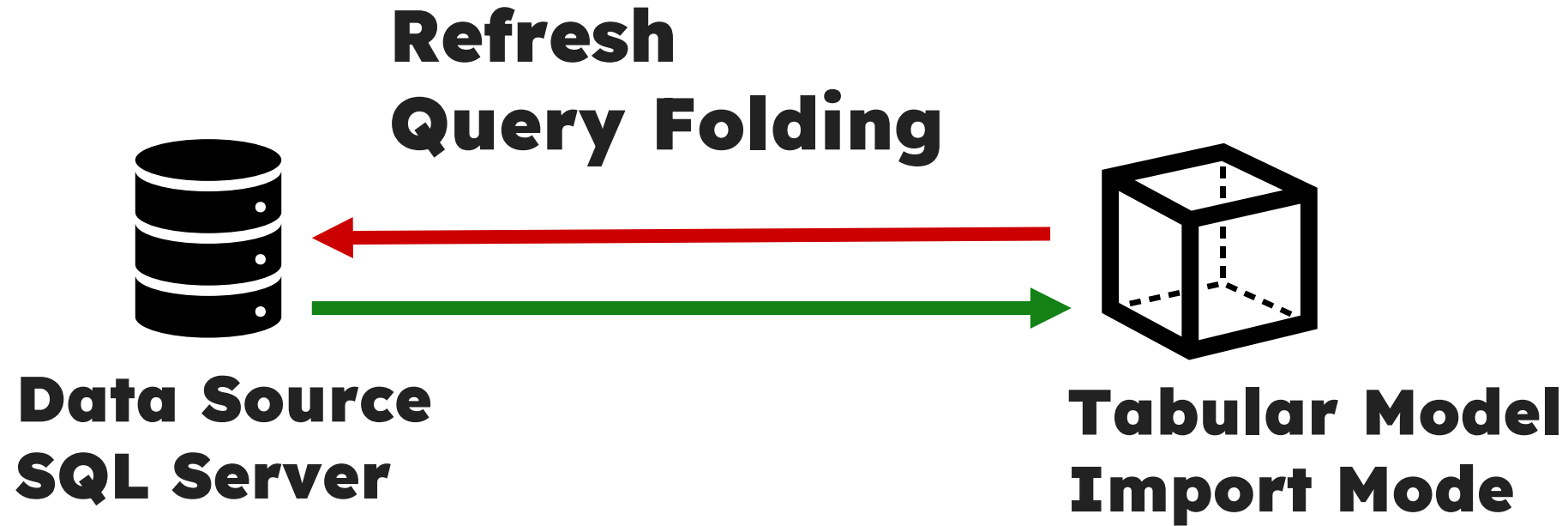
Storage Engine

- Ability to cache
- Ability to Multithread
- Operations depend on the storage engine
 - Vertipaq is very limited
 - But optimized

Vertipaq (Import)

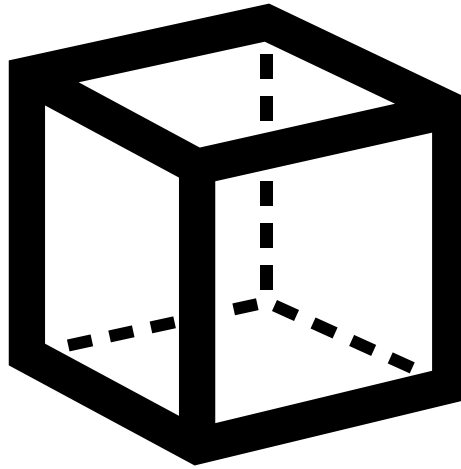
- Column Based
- Encoded
- Compressed

Step 1: Refresh



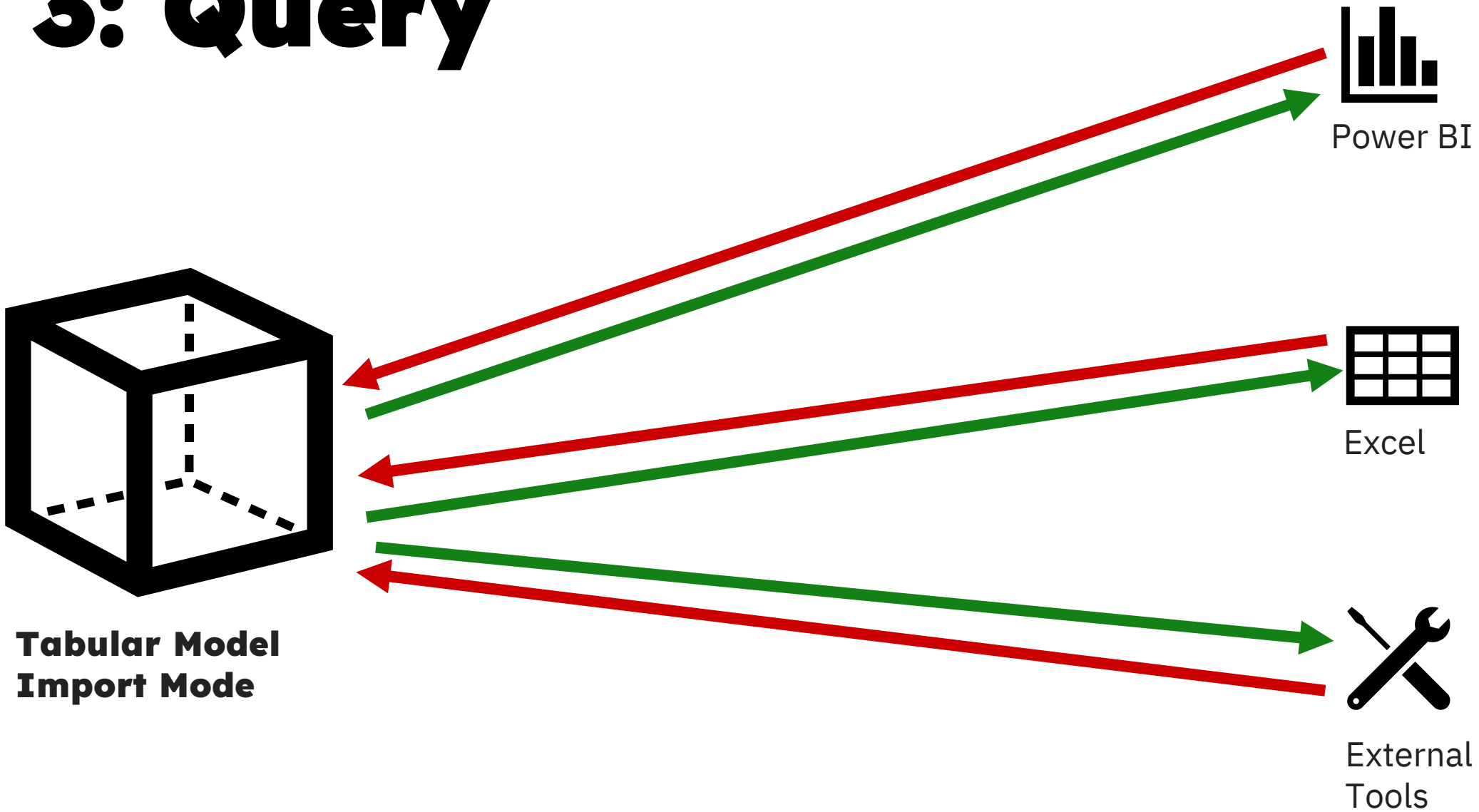
Step 2: Process

Processed



**Tabular Model
Import Mode**

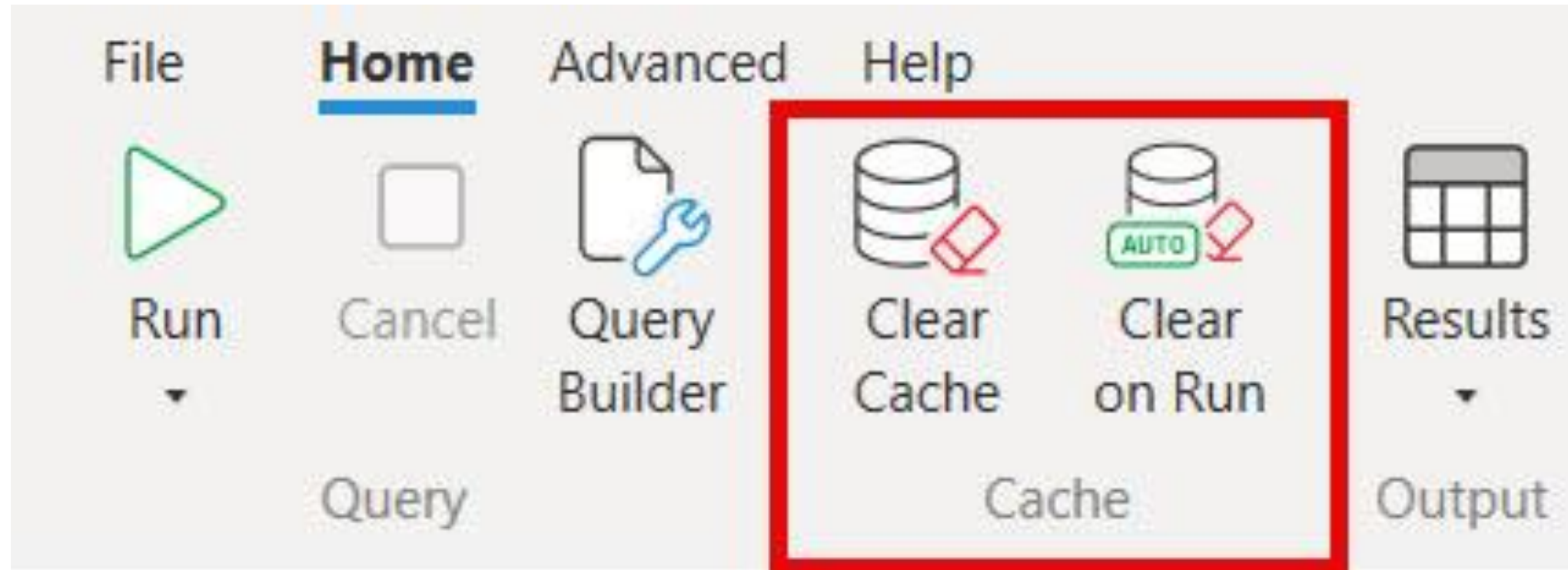
Step 3: Query



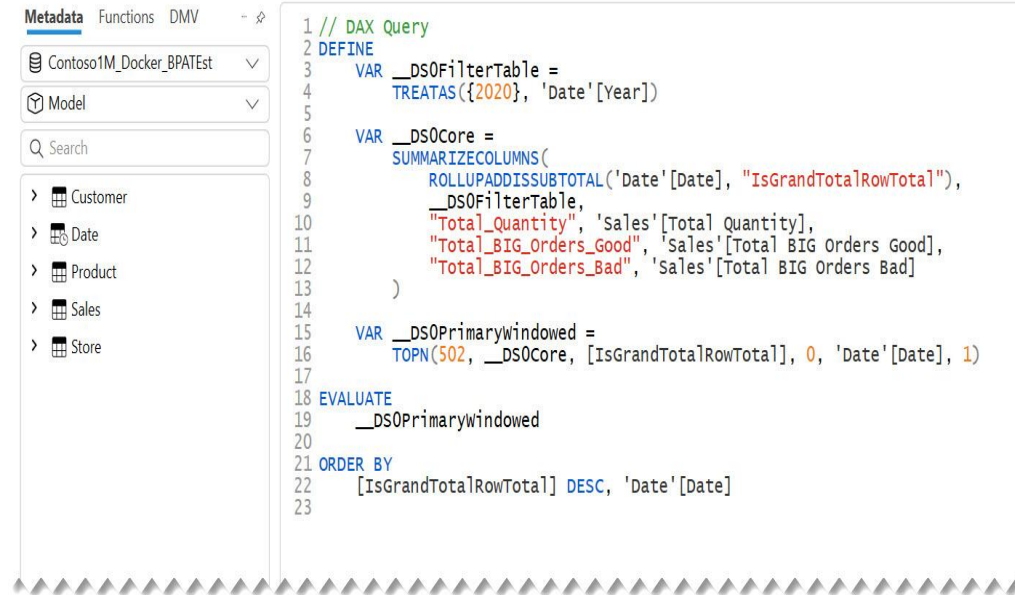
THAT'S BEYOND THE SCOPE...



Cache

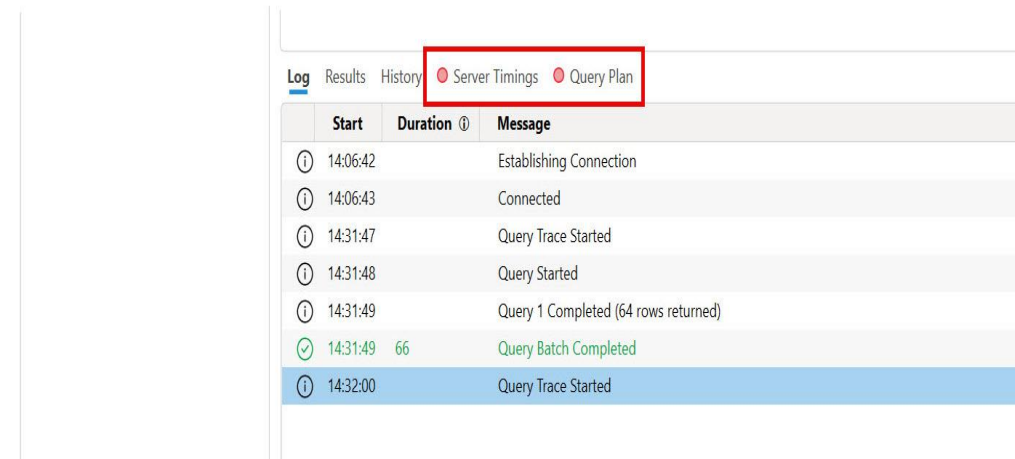


Location of Query and Server Timings



The screenshot shows the SQL Server Data Tools (SSDT) interface. On the left, the 'Model' browser displays a hierarchy: Contoso1M_Docker_BPATest > Model > Customer > Date > Product > Sales > Store. The main query editor displays a DAX query:

```
1 // DAX Query
2 DEFINE
3     VAR __DS0FilterTable =
4         TREATAS({2020}, 'Date'[Year])
5
6     VAR __DS0Core =
7         SUMMARIZECOLUMNS(
8             ROLLUPADISSUBTOTAL('Date'[Date], "IsGrandTotalRowTotal"),
9             __DS0FilterTable,
10            "Total_Quantity", 'Sales'[Total Quantity],
11            "Total_BIG_Orders_Good", 'Sales'[Total BIG Orders Good],
12            "Total_BIG_Orders_Bad", 'Sales'[Total BIG Orders Bad]
13        )
14
15     VAR __DS0Primarywindowed =
16         TOPN(502, __DS0Core, [IsGrandTotalRowTotal], 0, 'Date'[Date], 1)
17
18 EVALUATE
19     __DS0Primarywindowed
20
21 ORDER BY
22     [IsGrandTotalRowTotal] DESC, 'Date'[Date]
23
```



The screenshot shows the 'Log' window in SSDT, which displays a list of events. The 'Server Timings' tab is selected, and the 'Query Batch Completed' event is highlighted. The table below shows the log entries:

	Start	Duration ⓘ	Message
ⓘ	14:06:42		Establishing Connection
ⓘ	14:06:43		Connected
ⓘ	14:31:47		Query Trace Started
ⓘ	14:31:48		Query Started
ⓘ	14:31:49		Query 1 Completed (64 rows returned)
✓	14:31:49	66	Query Batch Completed
ⓘ	14:32:00		Query Trace Started

DAX – Return Product Table

EVALUATE

'Product'

Storage Engine Query

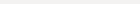
```
1 EVALUATE
2 'Product'
```

100 ▾

Log Results History **Server Timings**

Record Pause Stop Clear Copy Export Info

Copy SE query Scan Cache Batch Internal

Total 37 ms	SE CPU 0 ms x0.0	Line	Subclass	Duration	CPU	Par.	Rows	KB	Timeline	Query
		2	Scan	1	0		2,517	148		SELECT 'Product'[RowNumber], 'Product'[ProductKey]
FE 36 ms	SE 1 ms									

SE Queries	SE Cache
1	0 0.0%

```
SET DC_KIND="AUTO";
SELECT
'Product'[RowNumber],
'Product'[ProductKey],
'Product'[Product Code],
'Product'[Product Name],
'Product'[Manufacturer],
'Product'[Brand],
'Product'[Color],
'Product'[Weight Unit Measure],
'Product'[Weight],
'Product'[Unit Cost],
'Product'[Unit Price],
'Product'[Subcategory Code],
'Product'[Subcategory],
'Product'[Category Code],
'Product'[Category]
FROM 'Product';
```

Estimated size: rows = 2,517 bytes = 151,020



Vertipaq - xmsQL

SELECT

```
'Product'[RowNumber],  
'Product'[ProductKey],  
'Product'[Product Code],  
'Product'[Product Name],  
'Product'[Manufacturer],  
'Product'[Brand],  
'Product'[Color],  
'Product'[Weight Unit Measure],  
'Product'[Weight],  
'Product'[Unit Cost],  
'Product'[Unit Price],  
'Product'[Subcategory Code],  
'Product'[Subcategory],  
'Product'[Category Code],  
'Product'[Category]
```

FROM 'Product';

Storage Engine Query - Timeline

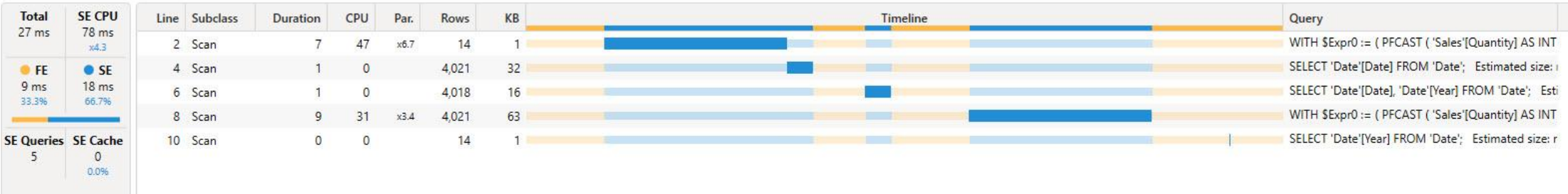
This one is mainly in FE.

Blue shows where SE comes into the timeline

Line	Subclass	Duration	CPU	Par.	Rows	KB	Timeline	Query
2	Scan	0	0		4,021	32		SELECT 'Date'[Date] FROM 'Date'; Estimated size: rows = 4,021 bytes = 32,168
4	Scan	0	0		4,021	32		SELECT 'Date'[Date] FROM 'Date'; Estimated size: rows = 4,021 bytes = 32,168
6	Scan	0	0		4,021	63		SELECT 'Date'[Date], MAX ('Date'[Date]) FROM 'Date'; Estimated size: rows = 4,021 bytes = 64,336
8	Scan	7	31	x4.4	4,021	63		WITH \$Expr0 := (PFCast ('Sales'[Quantity] AS INT) * PFCast ('Sales'[Net Price] AS INT)) SELECT 'Date'[Date], SUM (@\$Expr0) FR

Storage Engine Query - Timeline

This one 33% in FE.
Blue shows where SE comes in the timeline.



Server Timings

Total Time
(ms)

Total
180 ms

SE CPU
297 ms

Storage Engine
CPU

Parallelism
x1.7

Formula Engine
(Yellow)

FE
5 ms
2.8%

SE
175 ms
97.2%

Storage Engine
(Blue)

Bar
(Yellow and Blue)

SE Queries
2

SE Cache
0
0.0%

Storage Engine
Cache

Storage Engine
Queries

Simple way to increase Query Time

Apply a Table Filter

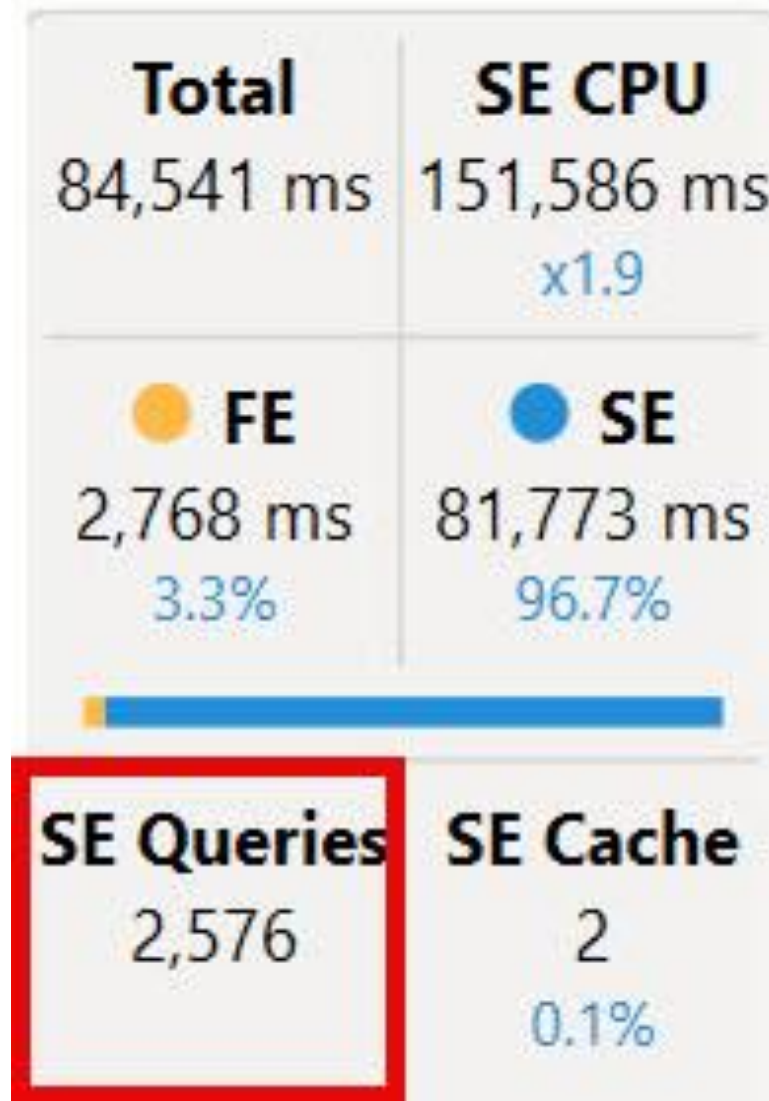
CALCULATE (

[Sales Amount],

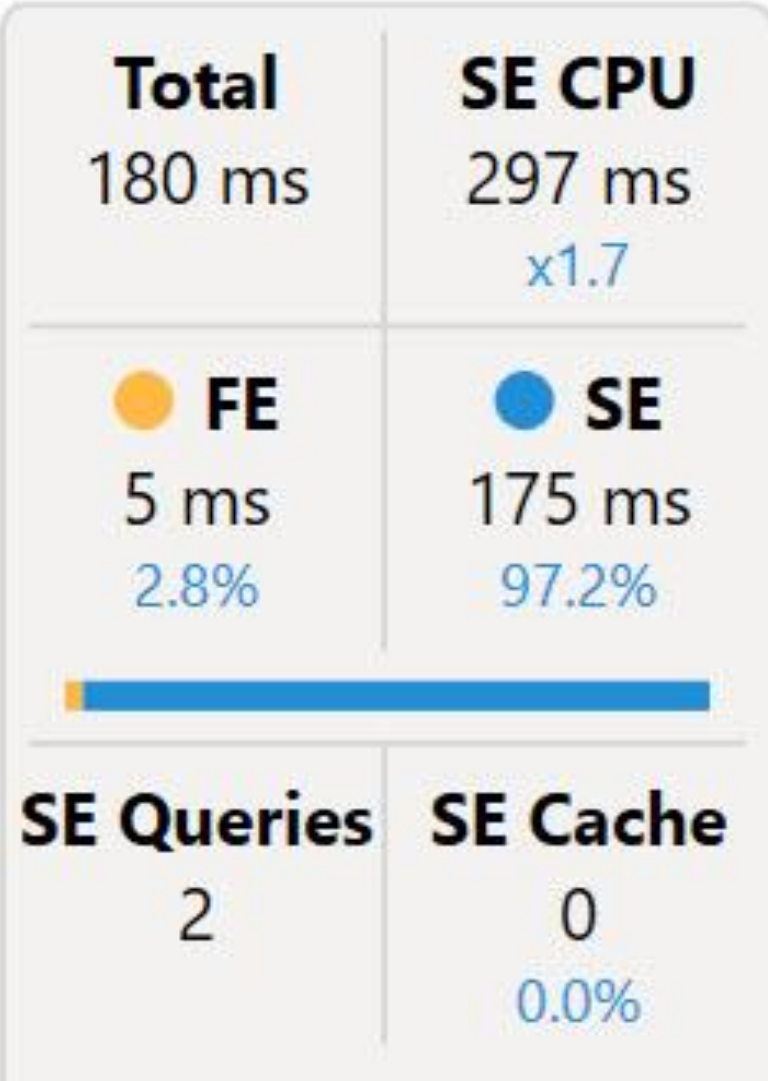
FILTER (Sales, Sales[Quantity] > 1)

)

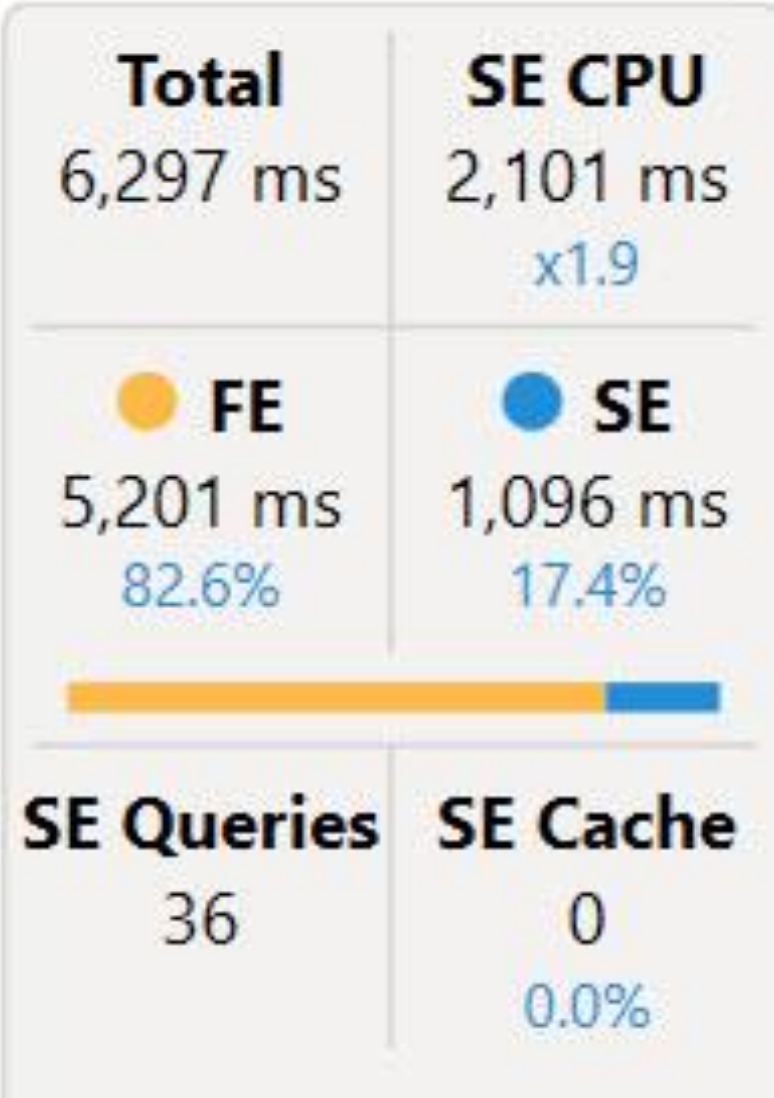
DAX Studio – Server Timings – Bad



DAX Studio – Server Timings - Good



DAX Studio – Heavy Formula Engine



Ways to invoke Formula Engine (Vertipaq)

- **IF Statements**
- **CONCATENATE**

CallbackDataID

WHERE

```
( COALESCE ( [CallbackDataID ( IF (
'Product'[Color] = "Blue", 1,0 ) ) ]
( PFDATAID ( 'Product'[Color] ) ) ) <> 0 ) ;
```

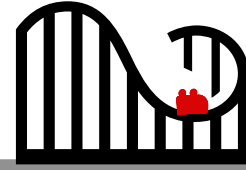
The more you know...

- If you can think like the engines
- You can anticipate performance issues

Power BI Demo

DAX Studio

Our Journey



1. Intro
2. Performance Analyzer
3. DAX Studio
- 4. Tabular Editor**
5. Conclusion

Proactive Measures*

(*no pun intended)

- How do we find issues with the Model and DAX ahead of time

**** pun is always intended**

Tabular Editor

- Main Uses
 - Develop the model
 - Make changes
 - Audit the model
 - Best Practice Analyzer

Tabular Editor 2.x

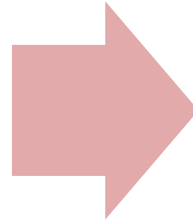
- Download
 - <https://github.com/TabularEditor/TabularEditor>
- Free version
- Version that was listed in DP-600 Study Guide

Tabular Editor 3.x

- Paid Version
- Extra Features
 - DAX Debugger
 - Script DAX

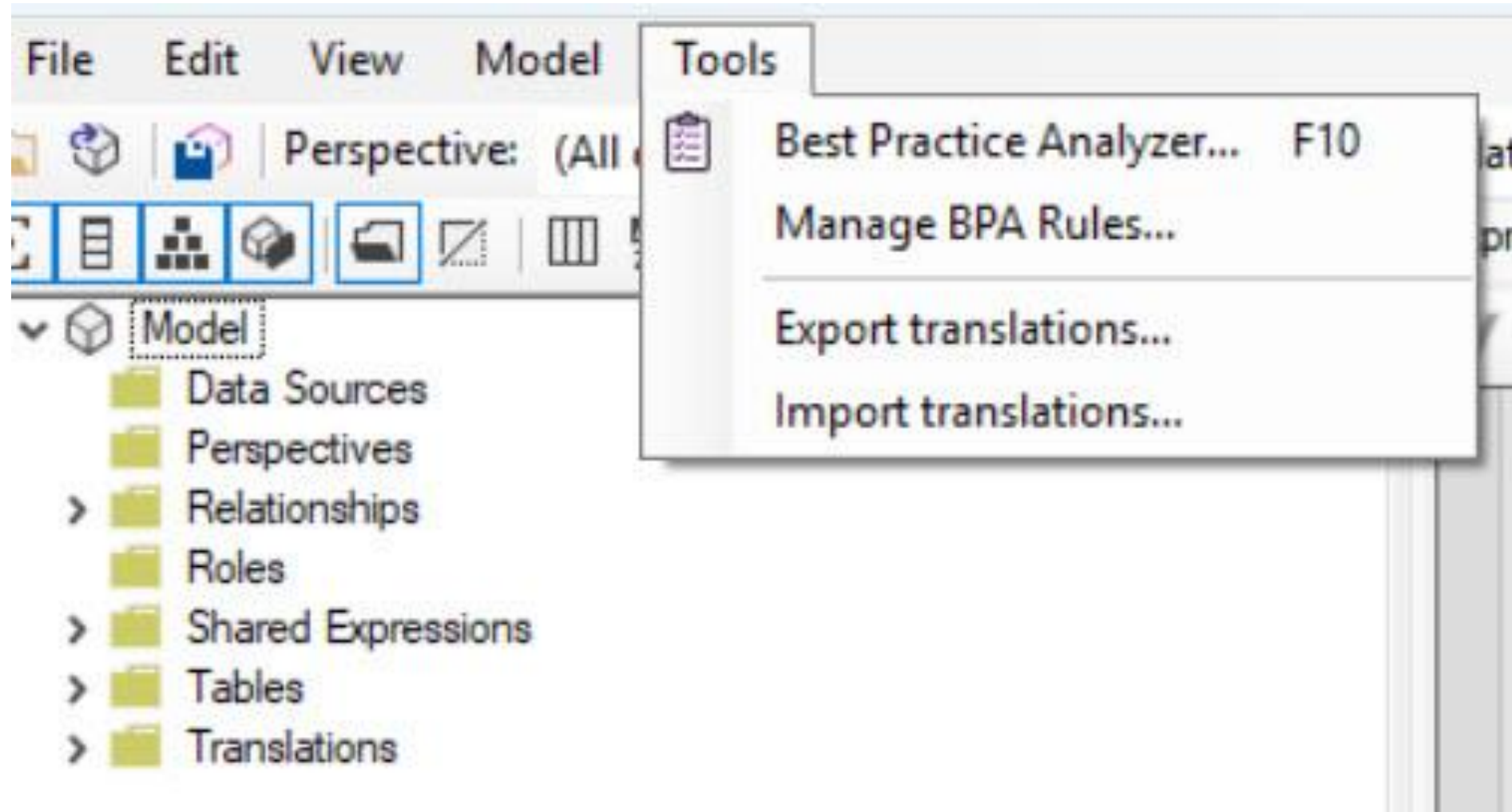
Tabular Editor 2

Optimize a semantic
model by using
Tabular Editor 2

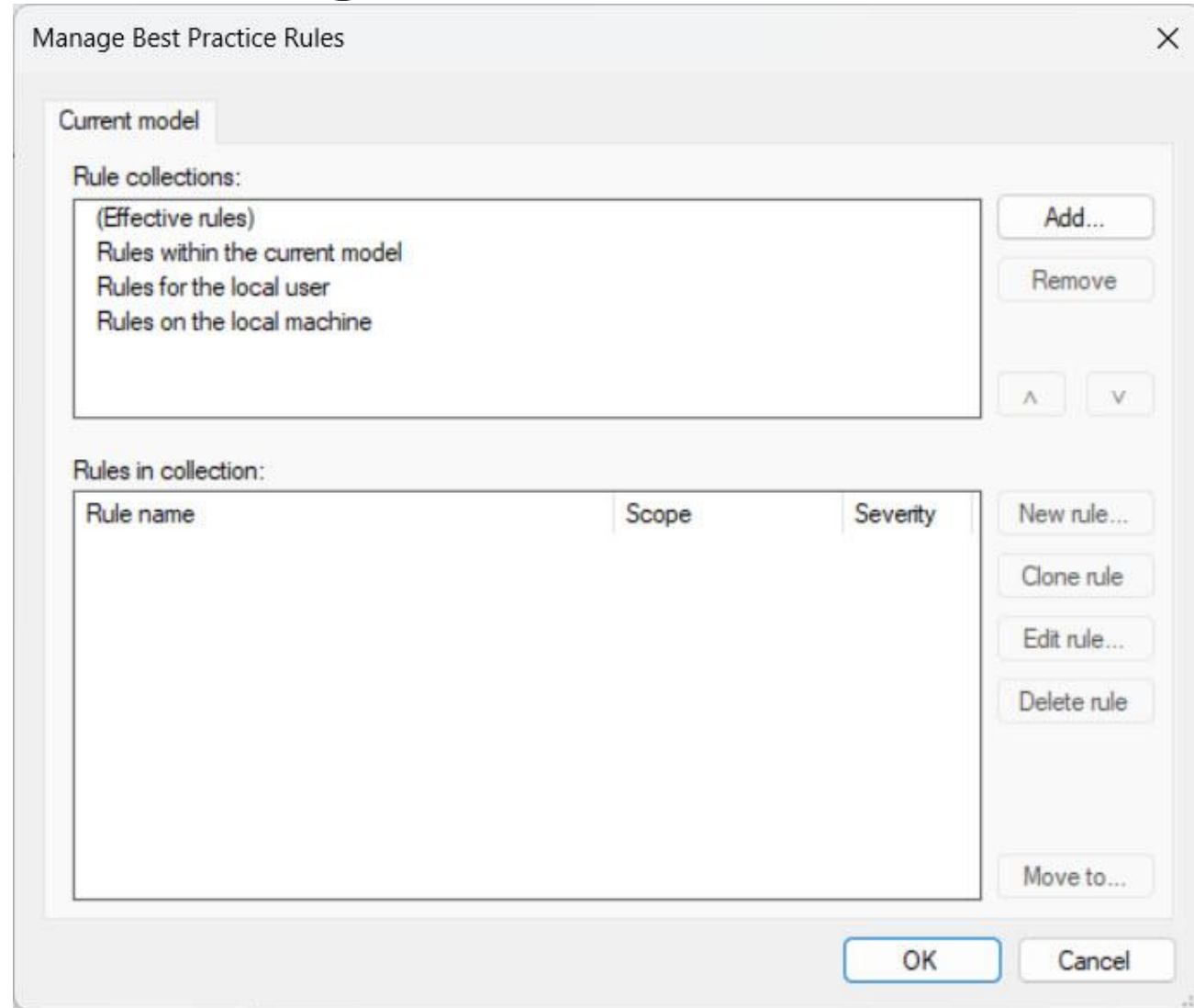


Best Practice Analyzer

Best Practice Analyzer Location



Add and Manage Rules



Best Practice Analyzer – C# Script Download

```
System.Net.WebClient w = new System.Net.WebClient();

string path = System.Environment.GetFolderPath(System.Environment.SpecialFolder.LocalApplicationData);
string url = "https://raw.githubusercontent.com/microsoft/Analysis-Services/master/BestPracticeRules/BPARules.json";
string version = System.Windows.Forms.Application.ProductVersion.Substring(0,1);
string downloadLoc = path + @"\TabularEditor\BPARules.json";

if (version == "3")
{
    downloadLoc = path + @"\TabularEditor3\BPARules.json";
}

w.DownloadFile(url, downloadLoc);

/*
// Italian
string url = "https://raw.githubusercontent.com/microsoft/Analysis-Services/master/BestPracticeRules/Italian/BPARules.json";

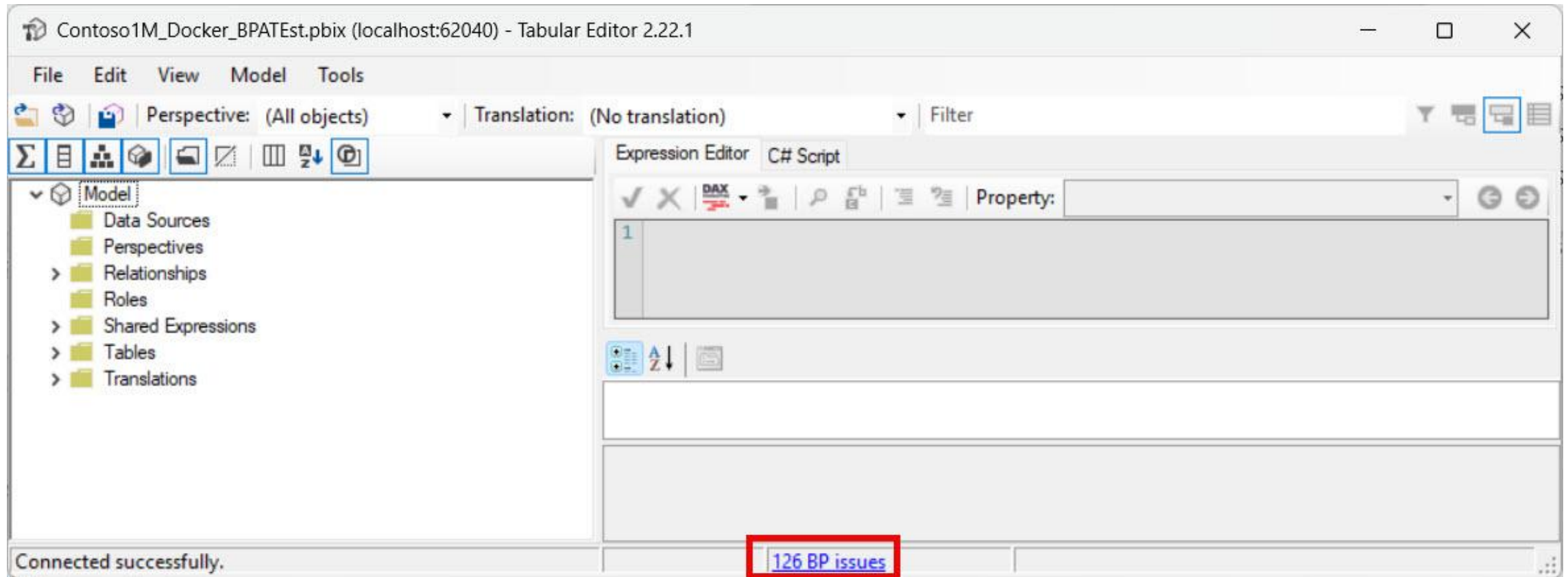
// Japanese
string url = "https://raw.githubusercontent.com/microsoft/Analysis-Services/master/BestPracticeRules/Japanese/BPARules.json";

// Spanish
string url = "https://raw.githubusercontent.com/microsoft/Analysis-Services/master/BestPracticeRules/Spanish/BPARules.json";
*/
```

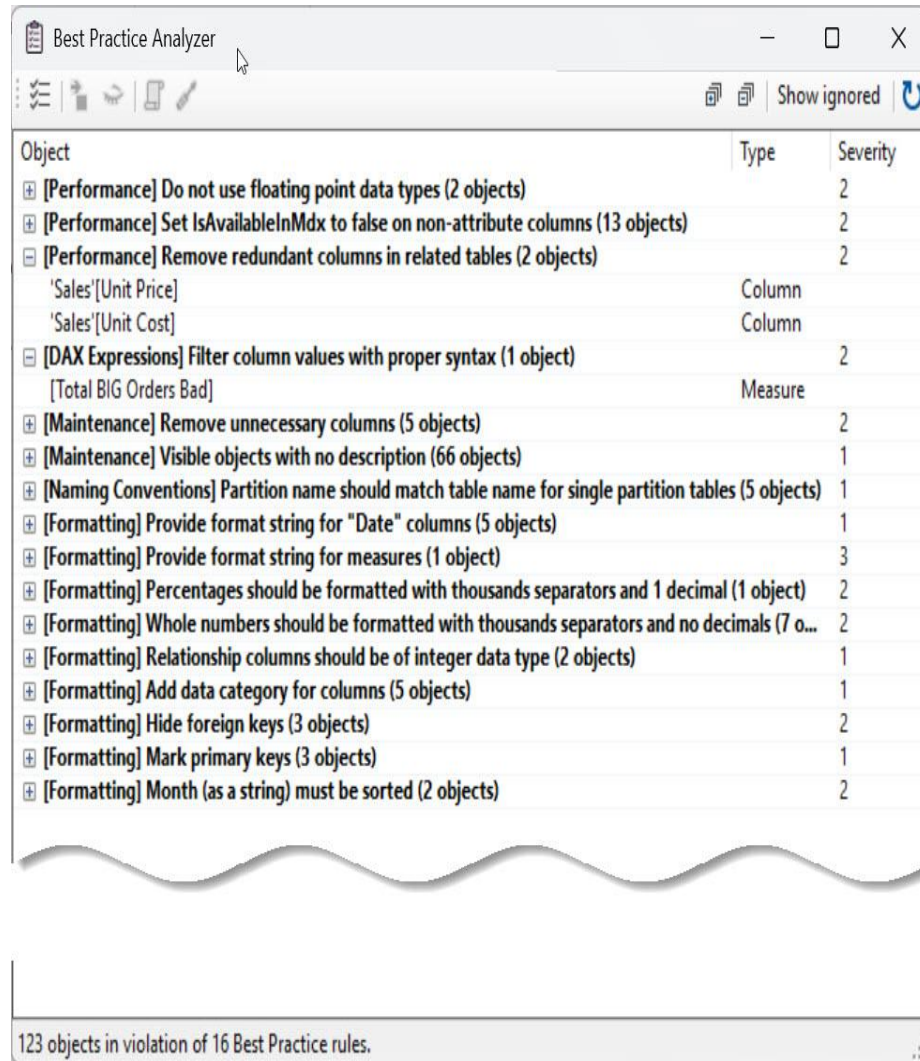
Nope – Not Running a Script

- Alternate Method
- Microsoft Analysis-Services – Best Practice Rules
 - <https://github.com/microsoft/Analysis-Services/tree/master/BestPracticeRules>
- Other languages besides English

BPA – Rule Exceptions for model



BPA – Rule Exceptions for model



Object	Type	Severity
+ [Performance] Do not use floating point data types (2 objects)		2
+ [Performance] Set IsAvailableInMdx to false on non-attribute columns (13 objects)		2
- [Performance] Remove redundant columns in related tables (2 objects)		2
'Sales'[Unit Price]	Column	
'Sales'[Unit Cost]	Column	
- [DAX Expressions] Filter column values with proper syntax (1 object)		2
[Total BIG Orders Bad]	Measure	
+ [Maintenance] Remove unnecessary columns (5 objects)		2
+ [Maintenance] Visible objects with no description (66 objects)		1
+ [Naming Conventions] Partition name should match table name for single partition tables (5 objects)		1
+ [Formatting] Provide format string for "Date" columns (5 objects)		1
+ [Formatting] Provide format string for measures (1 object)		3
+ [Formatting] Percentages should be formatted with thousands separators and 1 decimal (1 object)		2
+ [Formatting] Whole numbers should be formatted with thousands separators and no decimals (7 o...		2
+ [Formatting] Relationship columns should be of integer data type (2 objects)		1
+ [Formatting] Add data category for columns (5 objects)		1
+ [Formatting] Hide foreign keys (3 objects)		2
+ [Formatting] Mark primary keys (3 objects)		1
+ [Formatting] Month (as a string) must be sorted (2 objects)		2

123 objects in violation of 16 Best Practice rules.

BPA – Manage Rules

DAX Expressions

Rules in collection:

Rule name	Scope	Severity
☐ DAX Expressions		
☒ [DAX Expressions] Avoid using '1-{x/y}' syntax	Measures,Calculated Columns,Calculation Items	2
☒ [DAX Expressions] Avoid using the IFERROR function	Measures,Calculated Columns	2
☒ [DAX Expressions] Column references should be fully qualified	Measures,KPIs,Table Permissions,Calculation Items	3
☒ [DAX Expressions] Filter column values with proper syntax	Measures,Calculated Columns,Calculation Items	2
☒ [DAX Expressions] Filter measure values by columns, not tables	Measures,Calculated Columns,Calculation Items	2
☒ [DAX Expressions] Inactive relationships that are never activated	Relationships	2
☒ [DAX Expressions] Measure references should be unqualified	Measures,Calculated Columns,Calculated Tables,KPIs,Calculation Items	3
☒ [DAX Expressions] Measures should not be direct references of other measures	Measures	2
☒ [DAX Expressions] No two measures should have the same definition	Measures	2
☒ [DAX Expressions] The EVALUATEANDLOG function should not be used in production models	Measures	1
☒ [DAX Expressions] Use the DIVIDE function for division	Measures,Calculated Columns,Calculation Items	2
☒ [DAX Expressions] Use the TREATAS function instead of INTERSECT for virtual relationships	Measures,Calculation Items	2

What about that slow pattern?



Filter by the Whole Table

```
CALCULATE (  
    [Sales Amount],  
    FILTER (Sales, Sales[Quantity] > 1)  
)
```

Edit Rule

Edit Best Practice Rule

Name: [DAX Expressions] Filter measure values by columns, not tables

ID: FILTER_MEASURE_VALUES_BY_COLUMNS Severity: 2 Category: DAX Expressions

Description: Option 1: FILTER(VALUES(Table[Column]),[Measure] > Value)
Option 2: FILTER(ALL(Table[Column]),[Measure] > Value)
Reference: <https://docs.microsoft.com/power-bi/guidance/dax-avoid-avoid-filter-as-filter-argument>

Applies to: Calculated Columns, Calculation Items, Measures satisfying the following criteria:

Rule Expression Editor

```
Regex.IsMatch(Expression, "(?i)CALCULATE\s*\(\s*[\^,]+\s*(?i)FILTER\s*\(\s*\"[A-Za-z0-9 _]+'*\s*,\s*\"[^\"]+\s*\")\s*\)\s*or\s*Regex.IsMatch(Expression, "(?i)CALCULATETABLE\s*\(\s*[\^,]*\s*(?i)FILTER\s*\(\s*\"[A-Za-z0-9 _]+'*\s*,\s*\"[^\"]+\s*\")\s*\)\s*"
```

Minimum Compatibility Level: CL 1200 (SQL Server 2016 / Azure AS)

OK Cancel

Description with help link

- Instead of using this pattern `FILTER('Table','Table'[Column]="Value")` for the filter parameters of a `CALCULATE` or `CALCULATETABLE` function, use one of the options below. As far as whether to use the `KEEPFILTERS` function, see the second reference link below.
- Option 1: `KEEPFILTERS('Table'[Column]="Value")`
- Option 2: `'Table'[Column]="Value"`
- Reference: <https://docs.microsoft.com/power-bi/guidance/dax-avoid-avoid-filter-as-filter-argument>
- Reference: <https://www.sqlbi.com/articles/using-keepfilters-in-dax/>

Another Source

9. Filtering a column

The [FILTER](#) function is often overused. Its main purpose is for filtering columns based on measure values. If you're just filtering a column value, there's generally no need to use this function. In fact, using it in that scenario often degrades performance.

Don't use this logic:

US Revenue = **CALCULATE** ([Revenue], **FILTER** ('Geography', 'Geography'[Area] = "United States"))

Option 1:

US Revenue 1 = **CALCULATE** ([Revenue], 'Geography'[Area] = "United States")

Option 2:

US Revenue 2 = **CALCULATE** ([Revenue], **KEEPFILTERS** ('Geography'[Area] = "United States"))

[Top 10 Power BI mistakes and their best practice solutions](https://www.elegantbi.com/post/top10bestpractices)
(<https://www.elegantbi.com/post/top10bestpractices>)

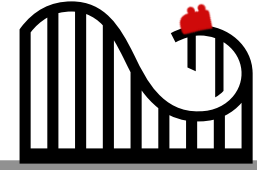
Change Description

- Link to company best practices
- Link to SharePoint that has more information

Power BI Demo

Tabular Editor

Our Journey



1. Intro

2. Performance Analyzer

3. DAX Studio

4. Tabular Editor

5. Conclusion

Key Takeaways for Optimizing Power BI Reports

- **Invest in Tools:** Utilize Performance Analyzer, DAX Studio, and Tabular Editor to enhance your Power BI models and reports.
- **Skill Enhancement:** These tools can significantly improve your skillset and productivity.
- **Overcome Limitations:** Power BI Desktop has its limitations, but these tools can help you overcome them and optimize your models and reports

Conclusion

- **Start with the Report:** Utilize Performance Analyzer to pinpoint areas that need optimization.
- **Leverage DAX Studio:** Employ DAX Studio to diagnose and improve slow DAX queries.
- **Optimize with Tabular Editor:** Use Tabular Editor to enhance the semantic model and ensure best practices are followed.

Resources



[Tabular Editor – Wonderful Training Free](#)



[Tabular Editor – Blog Posts](#)



[DAX Studio](#)



[Tabular Editor 2.x](#)

Resources



[Data Goblins - Sample Datasets](#)



[Data Mozart - Lots of DP-600 Resources](#)

[Visual Speed](#)

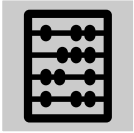


[The Definitive Guide to DAX - 2nd Edition](#)



[Optimizing DAX Book 2nd Edition](#)

Resources



[Elegant BI Blog \(Excellent Source\)](#)



[Microsoft Best Practice Rules Blog](#)



[Microsoft Github Best Practice Rules](#)



[DP-600 Exam Prep Book](#)

Thank you



Jason Romans
thedaxshepherd@gmail.com
www.thedaxshepherd.com

